ALZENNA BUNGA RACHEL
24 / 535355 / TK / 59380

Problem 1 : Algorithm Analysis

- Snippet A

```
//n >= 1
int sum = 0;
for (int i = 0; i<n; i++) {
    for (int j = 0; j < i; j++) {
        sum += arr[i] + arr[j];
    }
}
```

Line	Code / operation	cost per exec.	Number of exec.
(a)	sum = 0 ;	1 (assign)	1
(b)	Outer init : i=0 ;	1 (assign)	1
(b)	Outer cond : i<n ;	1 (comp)	A + 1
(b)	Outer incr : i++ ;	1 (arith) + 1 (assign)	A
(c)	Inner init : j = 0 ;	1 (assign)	A
(c)	inner cond : j<i ;	1 (comp)	A
(c)	inner incr : j++ ;	1 (arith) + 1 (assign)	B + A
(d)	Body = arr [i], arr [j]	1 (arith) + 1 (assign)	B
(d)	arithmetic (+)	1	B
(d)	assignment to sum	1	B

$T(n) = 1 + 1 + (A+1) + 2A + A + A + (B+A) + 2B + B + B$

$= 3 + 4A + 6B$

$\boxed{T(n) = 3n^2 + n + 3}$

Big-O class $= O(n^2)$

Verification : $(n = 4) = 3(4)^2 + 4 + 3$

$= 48 + 4 + 3 = 55$ Ops

Best / worst case

↳ jumlah iterasi tidak bergantung pada isi array, karena struktur loop sepenuhnya ditentukan oleh n, bukan isi data.

- Snippet B

```
//n >= 1 ; arr is a sorted array of n integers
int lo = 0, hi = n-1, target = 42 ;
bool found = false ;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2 ;
    if (arr [mid] == target) { found = true ; break ; }
```

GELATIK

ALDENNA BUNGA RACHEL
24 1535355 / TK 159380

```
else if (arr[mid] < target) {lo = mid + 1;}
else                        {hi = mid - 1;}
```

↳ times to halve n before reaching 1, $T(n)$ in terms of $[\log_2 n] + 1$

Line	Code/ Operation	cost per exec.	Executions
(a)	lo = 0	1	1
(a)	hi = n-1 (arithmetic + assign)	2	1
(a)	target = 42	1	1
(b)	found = false	1	1
(e)	comparison ==	1	A
(f)	arr [mid]	1	A
(f)	comparison <	1	A
(g)	mid + 1 or mid - 1	1	A
(g)	assignment (lo or hi)	1	A

.) $T(n) = 5 + (A+1) + 10A$

$= 6 + 11A,$

substitusi $A = [\log = n] + 1$

$(n) = 6 + 11([\log 2\, n] + 1)$

Big-O $= O(\log n)$

Best case $= O(1)$ → langsung ketemu di tengah

worst case $= O(\log n)$

Snippet C

```
// n >= 1
int count = 0;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        for (int k = 0; k < n; k += 2) {  // note : k += 2
            if (arr[i] != arr[j] && arr[k] > 0)
                count ++;
        }
    }
}
```

↳ the innermost loop runs $[\frac{n}{2}]$ times regardless of i and j. the middle loop runs a total of $\frac{n(n-1)}{2}$ times (summed over all i)

↳ $A = n$, $B = \frac{n(n-1)}{2}$, $C = [\frac{n}{2}]$, $D = B \cdot C = \frac{n(n-1)}{2} \cdot \frac{n}{2} = \frac{n^3}{4} - \frac{n^2}{4}$

ALZENNA BUNGA RACHEL
24/535355/TK/59380

Line	Operation / code	cost per exce.	Executions
(a)	count = 0	1	1
(b)	outer init	1	1
(b)	outer cond	1	A + 1
(b)	outer incr	1	A
(c)	middle init	2	A
(c)	middle cond	1	B + A
(c)	middle incr	1	B
(d)	inner init	1	B
(d)	inner cond	1	0 + B
(d)	inner incr (k += 2)	2	0
(e)	arr [i]	1	0
(e)	arr [j]	1	0
(e)	!=	1	0
(e)	arr [k]	1	0
(e)	>	1	0
(f)	count ++	1	0

$T(n) = 60 = 6\left(\frac{n^3}{4} - \frac{n^2}{4}\right)$

- $T(n) = \frac{3}{2}n^3 - \frac{3}{2}n^2 + O(n^2)$
- Big-O class = $O(n^3)$
- Best case = tidak berbeda (loop selalu penuh)
- Worst case = $O(n^3)$

GELATIK

ALZENNA BUNGA RACHEL
24 1835355 / TK 159380

※ Problem 4 : Amortized Analysis of PushBack

Part A – Trace

↳ for each call, record : the value pushed, the resulting size, the resulting capacity & cost of that specific call (cost = 1 for normal append ; cost = 1 + old size when a resize occurs).

call ※	value pushed	size after	capacity after	cost
1	10	1	2	1
2	20	2	2	1
3	30	3	4	1 + 2 (resize)
4	40	4	4	1
5	50	5	8	1 + 4 (resize)
6	60	6	8	1
7	70	7	8	1
8	80	8	8	1
9	90	9	16	1 + 8 (resize)
10	100	10	16	1
11	110	11	16	1
12	120	12	16	1

starting from capacity 2

Part B – Total cost

① Sum all values in the cost column. call this T_{12}

$1 + 1 + 3 + 1 + 5 + 1 + 1 + 1 + 9 + 1 + 1 + 1 = 26$

② Average cost per call ($T_{12}/12$)

$\frac{26}{12} \approx 2.17$

③ C_n (n-th cost), full formula for the total copy work done across all resizes during n pushbacks. identify & evaluate geometric series. Amortised cost per call is:

↳ total copy work occurs at sizes $2, 4, 8, \ldots, 2^k$

total cost $T(n) = n + \sum_{i=0}^{\log_2 n} 2^i$

Geometric series sum $\sum_{i=0}^{k} 2^i = 2^{k+1} - 1 \approx 2n$

total $T(n) \approx n + 2n = 3n$

Amortized cost $T(n)/n = 3$, which is $O(1)$

Part C – Growth factor comparison

↳ PushBack grows by adding a fixed amount $k = 4$ (new capacity = old capacity +4)

① Redo trace from part A (12 calls, starting from capacity 2) under new strategy, same 4 columns.

② comparing total cost of both strategies

③ derive $T(n)$ for both the doubling & the fixed increment strategies.

doubling → $O(n)$

fixed increment → $O(n^2)$ total.

} Amortised cost / call

ALZENNA BUNGA RACHEL
24/535355 MK 159380

Growth Factor Comparison (fixed increment) $k=4$

- start capacity $=2$, new capacity $=$ old $+4$
- Resizes occur when size hits : 2, 6, 10, 14, ...

①

value	size	capacity	cost
10	1	2	1
20	2	2	1
30	3	6	$1+2=3$
40	4	6	1
50	5	6	1
60	6	6	1
70	7	10	$1+6=7$
80	8	10	1
90	9	10	1
100	10	10	1
110	11	14	$1+10=11$
120	12	14	1

② comparison :

- total cost fixed (12 calls)

  $\hookrightarrow 1+1+3+1+1+1+7+1+1+1+11+1 = 30$
- total cost doubling (12 calls) $= 26$
- fixed increment is already more expensive.

③ Derivation :

- Doubling : Resizes at 2, 4, 8, ... total copies $\approx n$.

  Total time $O(n)$. Amortized $O(1)$
- fixed increment : Resizes at 2, 6, 10, ..., $4m+2$

  number of resizes $\approx n/4$

  cost of resizes at step m is proportional to current capacity ($\approx 4m$).

  total cost $T(n) = \sum_{m=1}^{n/4} 4m \approx 4 \cdot \frac{(n/4)^2}{2}$

  $\approx O(n^2)$
- Amortized cost : $O(n^2)/n = O(n)$
- Conclusion : Doubling is superior because it keeps amortized konstan cost, fixed increment makes amortized cost linear.

GELATIK